

Set 4 - HashMap and LinkedHashMap

Q1.

Create a HashMap in Java to store 5 food delivery order IDs (String) as keys and their order amounts (Double) as values, then print all key-value pairs using an iterator.

Answer:

```
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;

public class OrderAmounts {
    public static void main(String[] args) {
        HashMap<String, Double> orders = new HashMap<>();
        orders.put("ORD1001", 349.50);
        orders.put("ORD1002", 210.00);
        orders.put("ORD1003", 599.99);
        orders.put("ORD1004", 120.75);
        orders.put("ORD1005", 480.00);

        Iterator<Map.Entry<String, Double>> it = orders.entrySet().iterator();
        while (it.hasNext()) {
            Map.Entry<String, Double> entry = it.next();
            System.out.println("Order ID: " + entry.getKey() + " -> Amount: Rs. " + entry.getValue());
        }
    }
}

/* Sample Output (order may vary since HashMap is unordered):
Order ID: ORD1001 -> Amount: Rs. 349.5
Order ID: ORD1002 -> Amount: Rs. 210.0
Order ID: ORD1003 -> Amount: Rs. 599.99
Order ID: ORD1004 -> Amount: Rs. 120.75
Order ID: ORD1005 -> Amount: Rs. 480.0
*/
```

Q2.

Build a LinkedHashMap to store 4 recent songs played on Spotify, where the key is the song title and the value is the artist name. Display the songs in the order they were added.

Answer:

```
import java.util.LinkedHashMap;
import java.util.Map;

public class RecentlyPlayed {
    public static void main(String[] args) {
        LinkedHashMap<String, String> recentSongs = new LinkedHashMap<>();
        recentSongs.put("Kesariya", "Arijit Singh");
        recentSongs.put("Blinding Lights", "The Weeknd");
        recentSongs.put("Apna Bana Le", "Arijit Singh");
        recentSongs.put("Flowers", "Miley Cyrus");

        System.out.println("Recently Played (in order added):");
        for (Map.Entry<String, String> entry : recentSongs.entrySet()) {
```

```

        System.out.println(entry.getKey() + " - " + entry.getValue());
    }
}

/* Sample Output:
Recently Played (in order added):
Kesariya - Arijit Singh
Blinding Lights - The Weeknd
Apna Bana Le - Arijit Singh
Flowers - Miley Cyrus
*/
// LinkedHashMap preserves insertion order, unlike HashMap.

```

Q3.

Given a HashMap of Flipkart product names (String) and their prices (Integer), write code to filter and print only those products with a price above 1000 using an iterator. Hint: Use `entrySet()` and `Iterator` to loop through the map and check each price.

Answer:

```

import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;

public class FlipkartPriceFilter {
    public static void main(String[] args) {
        HashMap<String, Integer> products = new HashMap<>();
        products.put("Wireless Mouse", 599);
        products.put("Bluetooth Headphones", 1499);
        products.put("Laptop Stand", 899);
        products.put("Smart Watch", 2499);
        products.put("Phone Case", 299);
        products.put("Mechanical Keyboard", 3299);

        System.out.println("Products priced above Rs. 1000:");
        Iterator<Map.Entry<String, Integer>> it = products.entrySet().iterator();
        while (it.hasNext()) {
            Map.Entry<String, Integer> entry = it.next();
            if (entry.getValue() > 1000) {
                System.out.println(entry.getKey() + " - Rs. " + entry.getValue());
            }
        }
    }
}

/* Sample Output:
Products priced above Rs. 1000:
Bluetooth Headphones - Rs. 1499
Smart Watch - Rs. 2499
Mechanical Keyboard - Rs. 3299
*/

```

Q4.

Write a Java function that takes a HashMap of Instagram usernames (String) and their follower counts (Integer), sorts the entries by follower count in descending order, and prints the top 3 users. Constraint: Do not use streams or third-party libraries for sorting.

Answer:

```
import java.util.HashMap;
import java.util.Map;
import java.util.ArrayList;

public class TopInstaUsers {

    public static void printTop3(HashMap<String, Integer> users) {
        // Copy entries into an ArrayList so we can sort manually
        ArrayList<Map.Entry<String, Integer>> list = new ArrayList<>(users.entrySet());

        // Simple bubble sort in descending order of follower count
        for (int i = 0; i < list.size() - 1; i++) {
            for (int j = 0; j < list.size() - 1 - i; j++) {
                if (list.get(j).getValue() < list.get(j + 1).getValue()) {
                    Map.Entry<String, Integer> temp = list.get(j);
                    list.set(j, list.get(j + 1));
                    list.set(j + 1, temp);
                }
            }
        }

        System.out.println("Top 3 Instagram users by followers:");
        int limit = Math.min(3, list.size());
        for (int i = 0; i < limit; i++) {
            System.out.println((i + 1) + ". " + list.get(i).getKey() + " - " + list.get(i).getValue() +
        }

        public static void main(String[] args) {
            HashMap<String, Integer> users = new HashMap<>();
            users.put("foodie_priya", 12000);
            users.put("travel_with_raj", 45000);
            users.put("fit_arjun", 8000);
            users.put("style_by_neha", 67000);
            users.put("gamerzone", 21000);

            printTop3(users);
        }
    }

    /* Sample Output:
    Top 3 Instagram users by followers:
    1. style_by_neha - 67000 followers
    2. travel_with_raj - 45000 followers
    3. gamerzone - 21000 followers
    */
}
```

Q5.

Explain in your own words how a HashMap in Java is similar to a database table, using an example from a Zomato order history (order ID as key, order details as value).

Answer:

A HashMap stores data as key-value pairs, and a database table stores rows that are usually looked up by a unique primary key - the two ideas map onto each other very naturally. In a Zomato order history table, the primary key would be the Order ID, and each row would hold the order details (restaurant name, items, amount, status). In a Java HashMap, we can represent the exact same thing: the Order ID is the key, and an OrderDetails object (or a String summary) is the value. Example: HashMap<String,

```
String> orderHistory = new HashMap<>(); orderHistory.put("ORD1001", "Domino's Pizza - Rs. 599 -  
Delivered");
```

Just like a database uses the primary key to instantly fetch a row without scanning the whole table, a HashMap uses the key's hash code to instantly locate the value without scanning every entry - both give very fast (close to $O(1)$) lookups. The main difference is that a HashMap lives in memory during the program's run, while a database table persists the data permanently on disk.